QuantSim Explained

What this is	A walkthrough of the whole QuantSim project — what it is, what it does, the ideas behind it, how it is built, and how it all works together
Who it's for	You (and your teammates) — so you can understand and explain every part of the project in your own words
Companion to	*QuantSim — Requirements (FR/NFR)* and *QuantSim — Architecture & Physical Design*, which are the precise, formal versions of everything summarized here
How to read it	Top to bottom the first time. After that, jump to whatever section you need. Each concept is explained from scratch, with the plain idea first and the precise version second

How to use this document

This is the friendly companion to your two heavy specification documents. The specs are written to be exact and defensible; this one is written to be *understood*. Wherever a spec says something in careful technical language, this document says the same thing in ordinary words, usually with a small analogy, and then tells you the proper term so you recognise it when a panel member uses it.

The order is deliberate. First the big picture (what are we even building, and why). Then the people who use it and the features they get. Then a section that teaches every concept the project relies on, from market simulation to the topology mathematics, assuming no prior knowledge. Then how the system is put together as software. Then several end-to-end "what actually happens when I click this" walkthroughs. Finally the technology list, a numbers-at-a-glance sheet, and a glossary you can skim the night before the defense.

You do not need to memorise anything to read this. Read it like a story about your own project.

Part 1 — The big picture

1.1 QuantSim in one breath

QuantSim is a platform where you can **build a realistic-but-fake stock market, run an investment strategy inside it, and then measure how risky that strategy really is** — using three different "risk lenses," one of which (the topology one) is genuinely novel — and it **explains its answers** instead of just spitting out numbers.

1.2 The slightly longer version, with the core analogy

Think of a **flight simulator**. Before a pilot ever flies a real plane full of passengers, they practise in a simulator that behaves like a real aircraft — including the dangerous situations you would never want to create on purpose, like an engine failure in a storm. The simulator is safe, repeatable, and you can throw disasters at it deliberately to see how the pilot copes.

QuantSim is a **flight simulator for investing**. Instead of a plane it has a *market* — the up-and-down movement of stock prices over time. Instead of a pilot it has a *trading strategy* — a set of rules for when to

buy and sell. And instead of "what happens if the engine fails," it asks "what happens to this portfolio if the market crashes, or if everything I own suddenly starts moving together, or if one sector collapses."

The point of the whole thing is **risk**: not "how much money could I make," but "how badly could this go wrong, and would I have seen it coming?" That last part — *would I have seen it coming* — is where the interesting, original part of the project lives.

1.3 The problem it actually solves

Here is the honest problem QuantSim is built around.

Most of the tools people use to measure market risk are **backward- or side-ways-looking**. They tell you that things are already dangerous once the danger has arrived. The classic example is *correlation*: when a crash hits, almost everything falls together, so the "everything is moving together" alarm goes off — but by then you are already in the crash. Volatility (how jumpy prices are) behaves the same way; it spikes *during* the storm, not before it. These are like a smoke alarm that only goes off once the room is already full of smoke. Useful, but late.

The bet behind QuantSim is that the market's **underlying structure** — the *shape* of how everything is connected — starts to shift *before* the obvious indicators light up. If you could watch that shape and notice it deforming, you might get an earlier warning. Measuring the "shape" of data is exactly what a branch of mathematics called **topology** is for, and that is the original idea at the centre of this project.

On top of that, two more everyday problems get solved:

Ordinary learning-and-backtesting platforms let you simulate prices, but they rarely tell you *why* a number came out the way it did — they are black boxes. And most platforms don't let you deliberately stage disasters; QuantSim's **stress testing** does exactly that, on purpose, so you can measure the damage before it happens for real.

1.4 The one big idea: three risk lenses over one market

If you remember only one diagram from this whole document, make it this one. QuantSim looks at risk through **three different lenses**, and they all sit on top of the same foundation — a *market environment* (real or synthetic prices) that everything else consumes.

1. **Stress testing** — the "what if a disaster happens" lens. Stage a flash crash, a volatility spike, a sector collapse, and measure how much the portfolio loses and how long it takes to recover.
2. **Topological structure analysis (TDA)** — the "is the shape of the market quietly changing" lens. This is the novel, research-grade part: watch the geometry of how assets are connected and flag when it deforms, hopefully earlier than conventional signals.
3. **Machine-learning prediction** — the "what does the data suggest is coming" lens. Forecast volatility, classify the current market regime, estimate the chance of a big drawdown, score a portfolio's overall health.

And wrapped around all three is **explainability**: every answer is attributed to its causes and narrated in plain language, so the platform is never a black box.

That is the entire product in one sentence: *a simulated market you can run strategies inside, examined through three risk lenses, with every answer explained.*

1.5 What's included, and what's deliberately left out

A big part of sounding credible in a defense is being clear about your own boundaries. QuantSim's scope is defined by what the team commits to actually demonstrating.

In scope (the committed core): everything needed for the central promise to work end to end — bringing in and cleaning market data, building synthetic markets from four stochastic models, running an event-driven trading engine, computing the full topology pipeline and the stress tests, producing the core ML predictions, explaining results with SHAP and plain-English text, and showing it all on a dashboard with alerts. In requirements terms this is the **64 "Core" functional requirements** (out of 81 total), all demonstrable on a single laptop-scale setup.

Explicitly out of scope (and you should say so proudly, not apologetically):

- **No live/real-money trading.** QuantSim never sends a real order to a real exchange. It is a decision-support and analysis tool. Everything happens against historical or simulated data.
- **No paid real-time market feed.** Data comes from free or academic sources, played back as "pseudo-real-time" replay. No expensive live tick feed.
- **No investment advice.** It does not tell anyone what to buy. It informs a human who makes the decision.

Stating these out loud matters: it stops the project from looking like it is pretending to be a real trading system, and it keeps the focus on the part that is genuinely hard and genuinely yours — the analytics and the correctness.

Part 2 — Who uses it, and what each person can do

QuantSim has **four kinds of user**, arranged as a ladder of increasing power. The cleanest way to think about them is by the *job* each one does, not by "who is more senior."

Viewer is the read-only user. They look at finished dashboards, replays, and alerts, and they cannot change anything in the system. Think of someone who has been given a link to watch the results.

Analyst can do everything a viewer can, *plus* run analysis on things that already exist: launch a stress test, run a topology analysis, compute strategy metrics, set up alert rules, and arrange their own dashboard. What they cannot do is create the raw ingredients — they don't write trading strategies or bring in new data.

Quant (short for quantitative analyst) can do all of that *plus* author the inputs to the simulation core: design trading strategies, calibrate models, launch simulation runs, create portfolios, and train and register machine-learning models. This is the "power user who builds the experiments" role.

Admin is the operator. On top of everything else, an admin manages user accounts, assigns roles, maintains the master list of tradable instruments, and configures the system. Importantly, admin is *not* a super-quant — it is a governance role. It holds the keys to run the platform, not necessarily to do the deepest analysis.

Two everyday analogies help. First, it is like a **newspaper**: readers (viewer) just read; researchers (analyst) dig through the archives and write internal memos; journalists (quant) create the stories; and editors/IT (admin) manage who works there and what tools exist. Second, the ladder is about *capability*, and on top of the role there is a separate rule about *which specific data* you're allowed to see — holding the analyst role does not mean you can see *everyone's* portfolios, only the ones you've been granted. That second rule is called **row-level scoping**, and it is checked wherever the data actually lives, so nobody can sneak past it by taking an unusual route into the system.

Part 3 — What the system provides (the features, grouped)

This part answers "what are we actually giving the user." It walks through the eight functional areas at the level of *features*, saving the deeper "how does that even work" for Part 4 and the "how is it built" for Part 5. These eight areas map one-to-one onto the eight services you'll meet later, which is a nice property — there is only one map to learn.

Market data and signal ingestion. This is the front door for information. It pulls in historical price bars (the open/high/low/close/volume of each instrument each day), financial news articles, and reference data about which instruments exist. Crucially it **cleans** everything before anyone else is allowed to see it — catching missing days, duplicate rows, impossible prices, and so on — and it computes the everyday derived numbers (returns, volatility, correlations) *once* so that nobody downstream recomputes them differently. It also owns the idea of a **market environment**: a thing a portfolio can "live inside," whether that's a replay of a real historical period or a synthetic simulation.

Stochastic market simulator. This manufactures *fake but realistic* markets. It takes a real instrument, fits a mathematical model to how that instrument has actually behaved, and then generates many possible futures from that model. "Stochastic" just means "involving randomness" — each generated future is one random roll of the dice, and you generate thousands to see the range of what could happen.

Event-driven trading engine. This is where a strategy meets a market. It processes events in strict time order — a price tick arrives, the strategy decides, an order is placed, it gets filled at a realistic price (including trading costs and slippage), and the portfolio updates. Its special property is that it keeps a **perfect, unerasable log** of every event, so you can replay a portfolio's entire history exactly, step by step.

Machine learning and financial intelligence. This turns data into predictions: a sentiment score for each news article, a volatility forecast, a market-regime classification, a drawdown-risk probability, an overall portfolio-health score, and strategy performance metrics like the Sharpe ratio. Behind these it keeps two serious pieces of infrastructure — a **model registry** (so every prediction can be traced to the exact model version that made it) and a **point-in-time feature store** (so a model can never accidentally "cheat" by using information from the future).

Stress testing and market-structure analysis (Risk & TDA). Two capabilities in one place. **Stress testing** stages deliberate catastrophes against a portfolio and reports the damage: maximum loss, value-at-risk, recovery time, which instruments hurt the most, and how concentrated the portfolio is. **Topological data analysis** runs the shape-of-the-market pipeline and flags when the structure shifts. This is the headline feature of the whole project.

Explainable financial reasoning. This makes sure nothing is a black box. It uses a technique called **SHAP** to say which inputs pushed a prediction up or down, identifies which assets drove a portfolio's gains or losses, and — via a large language model — writes a plain-English sentence explaining a prediction, an alert, or a stress result. If the language model is unavailable, it falls back to a templated explanation rather than failing.

Analytics dashboard and alerting. This is the single screen over everything: portfolio overviews, trade replays, risk heatmaps, volatility charts, drawdown charts, stress results, regime views, and the topological-risk view. It also runs an **alerting** engine that watches conditions on your behalf and pushes you a notification the moment something crosses a threshold — and it is careful to alert you **once per condition, not once per event**, so it never floods you.

Platform, identity and governance. The quiet but essential plumbing that the other seven all rely on: user accounts, the four roles, logging people in, deciding what each person is allowed to do, and keeping an **audit trail** that records every change (including what a deleted thing used to be). This area has no glamorous feature of its own; its job is to make the whole platform trustworthy and accountable.

Part 4 — The concepts, explained from scratch

This is the longest part on purpose. It teaches every idea QuantSim relies on, assuming you know nothing about finance or the mathematics going in. Read it once slowly. Each concept follows the same shape: the plain idea, then a small analogy, then the proper term and just enough precision to defend it.

4.1 The raw material: prices, returns, volatility, correlation

Everything starts with a **price series** — a list of what an instrument was worth at each point in time. A "bar" of price data usually has five numbers for a period (say a day): the **open** (first price), **high**, **low**, **close** (last price), and **volume** (how much was traded). This is called **OHLCV** data.

You almost never work with raw prices, though. You work with **returns** — the *percentage change* from one period to the next. If a stock goes from 100 to 102, that's a +2% return. Why the switch? Because a 2 rupee move means something completely different on a 100 rupee stock than on a 10,000 rupee stock, but +2% is comparable across everything. Returns put all instruments on the same footing.

Volatility is how *jumpy* those returns are — the statistical spread (standard deviation) of returns. Low volatility means the price drifts along gently; high volatility means it lurches around. Volatility is the single most common everyday measure of "risk" because a jumpy asset can hurt you fast. The key limitation, as mentioned earlier, is that volatility tells you about danger mostly *while* it's happening.

Correlation measures whether two instruments *move together*. It runs from +1 (they rise and fall in perfect lockstep) through 0 (no relationship) to −1 (when one rises the other falls). Correlation is the foundation of **diversification**: if you hold ten things that all move together, you effectively hold one thing ten times and you are not diversified at all. The dangerous phenomenon in real crashes is that correlations *spike toward +1 exactly when you need them low* — everything falls together — which is precisely the late-warning problem QuantSim's topology lens is trying to beat.

QuantSim computes returns, volatility and a **correlation matrix** (every instrument against every other) centrally, once, so that all three risk lenses are looking at the same numbers.

4.2 Building a fake market: stochastic models

To stress-test a strategy against situations that haven't happened yet, you need to *manufacture* plausible price paths. That's what a **stochastic model** does — "stochastic" is just the mathematician's word for "having randomness built in." Each model is a recipe that produces one random-but-realistic future price path; run the recipe thousands of times and you get thousands of possible futures. QuantSim implements **four** such models, and it's worth knowing what each one is *for*, because they capture different real-world behaviours.

Geometric Brownian Motion (GBM) is the classic baseline. The idea: prices drift upward (or downward) at some average rate, with random wobble layered on top, and the wobble is proportional to the price. It's called "geometric" because it works on percentage changes, which keeps prices positive (a real stock can't go negative). This is the same mathematics underneath the famous Black–Scholes option-pricing model.

Analogy: a leaf floating down a gently sloping river — steady drift downstream, plus random jostling from the current. Its weakness is that it's *too* smooth: real markets jump more than GBM allows.

Jump-diffusion fixes exactly that weakness. It's GBM plus occasional sudden **jumps** — to model the reality that prices sometimes gap violently on news (an earnings shock, a geopolitical event). Analogy: the same leaf on the river, but every so often someone throws a rock in and the leaf lurches. This is how you generate the "flash crash" kind of behaviour.

Ornstein–Uhlenbeck (OU) is a **mean-reverting** model: instead of drifting away forever, the value is pulled back toward a long-run average, like a spring. This suits things that shouldn't wander off indefinitely — interest rates, or the price *gap* between two related assets in a pairs-trading strategy. Analogy: a ball attached to a spring; push it away and it gets tugged back toward the resting point, with random nudges along the way.

Regime-switching captures the fact that markets have **moods**. Sometimes the market is calm (low volatility, gentle drift); sometimes it's in crisis (high volatility, sharp moves). A regime-switching model has two or more of these "regimes" and randomly flips between them according to set probabilities. Analogy: weather that switches between "calm season" and "storm season," each with its own character, with some chance each day of the season changing. This is the model most directly connected to the idea that the market's *state* can change — which is the same intuition the topology lens chases from a different mathematical direction.

Calibration is the step that makes any of these realistic: you *fit* the model's knobs (drift, volatility, jump frequency, spring strength) to a real instrument's actual history, so the fake market resembles the real one it's based on. A rule QuantSim enforces: you can only calibrate a model on instruments it's actually allowed to represent — you can't fit an interest-rate model to a tech stock and pretend it's valid.

4.3 Rolling the dice thousands of times: Monte Carlo

Once you have a model that can generate one random future, the natural question is "okay, but what's the *range* of what could happen?" The answer is **Monte Carlo simulation**: run the random recipe a very large number of times, collect all the outcomes, and look at the distribution.

The name comes from the Monte Carlo casino — it's the mathematics of learning something by repeated random trials. Analogy: you want to know the chance of rolling a total of 7 with two dice. You could work out the probability by hand, or you could just roll the dice ten thousand times and count. Monte Carlo is the "just roll them ten thousand times" approach, applied to markets that are far too complicated to solve with a formula.

The beautiful practical property here is that each simulated path is **completely independent** of the others — path number 5,000 doesn't need to know anything about path number 4,999. That means you can compute them all *at the same time* on different workers. In the jargon this is called **embarrassingly parallel**, and it's the reason QuantSim's simulation engine can hand chunks of the work to a pool of background workers and get a big speed-up. When you hear "we shard the Monte Carlo paths across workers," this is what it means.

4.4 Lens one: stress testing

Stress testing is the most intuitive of the three risk lenses. You take a portfolio and deliberately subject it to a nasty, pre-defined scenario, then measure the damage. It answers "if *this specific bad thing* happened tomorrow, how much would I lose, and how long until I recover?"

QuantSim stages several named scenario types: a **flash crash** (a sudden severe drop), a **liquidity shock** (it becomes hard to sell without moving the price against you), a **volatility spike** (everything gets violently jumpy), a **sector collapse** (one industry falls apart), and a **correlated failure** (the diversification benefit vanishes because everything moves together). Each scenario is applied to the portfolio and the platform reports a consistent set of outcomes: the maximum loss, the value-at-risk and conditional value-at-risk under that scenario (defined next), the recovery time, and which individual instruments contributed most of the pain.

The value of doing this is that it's *forward-looking by construction*. You don't wait for a crash; you conjure one and study it in safety. That's the flight-simulator idea made concrete.

4.5 The risk numbers: VaR, CVaR, drawdown, concentration

A few specific risk measures show up all over the platform, so it's worth understanding each one plainly.

Value at Risk (VaR) answers: "on a normal-to-bad day, what's the most I'd expect to lose?" More precisely, the 95% one-day VaR is the loss level that only the worst 5% of days exceed. Analogy: it's the "I'm 95% confident I won't lose more than X today" line. Its famous flaw is that it says *nothing about how bad the other 5% get* — it marks the door to the bad room but won't tell you what's inside.

Conditional Value at Risk (CVaR), also called Expected Shortfall, fixes that flaw: it's the *average* loss *given* that you're in that worst 5%. It answers "and when it does go past the VaR line, how bad is it on average?" CVaR is the more conservative, more honest measure because it looks squarely at the tail. QuantSim reports both, and knowing why CVaR is preferred (it looks *inside* the bad room, and it behaves more sensibly when you combine portfolios) is an easy point to make in defense.

Drawdown is the drop from a peak to a subsequent trough — "from its highest point, how far down did it go before recovering?" **Maximum drawdown** is the worst such fall over a period. This is the measure that captures the *lived experience* of pain, because investors feel losses from the high-water mark, not from where they started.

Risk concentration measures whether your risk is dangerously bunched up — too much in one instrument, one sector, or one hidden common factor. A portfolio can *look* diversified (many holdings) while being concentrated (they all secretly depend on the same thing). QuantSim surfaces concentration explicitly so this hidden fragility is visible.

4.6 Lens two: topology and TDA — the original idea

This is the centrepiece, so it gets the fullest treatment. Take it slowly; the payoff is being able to explain your project's differentiator with real confidence.

4.6.1 What "topology" even means

Topology is the branch of mathematics that studies the *shape* of things — specifically, the properties that survive stretching and bending. A topologist famously "can't tell a coffee mug from a doughnut," because each has exactly one hole (the mug's handle, the doughnut's centre) and you could mould one into the other without tearing. What topology cares about is not distance or angle but connectedness: how many pieces, how many holes, how many voids.

The leap QuantSim makes is this: **a market can be treated as a shape.** If you place every instrument as a point in space, positioned so that instruments that move together sit close and instruments that move

independently sit far apart, then the whole market becomes a *cloud of points* — and that cloud has a shape. And the claim is that the *shape changes character before a crash* — points that were spread out start clumping, loops that were open start closing — even while the plain averages still look calm.

4.6.2 Turning a market into a shape (the point cloud)

The bridge from prices to shape is correlation. Start with the correlation matrix from §4.1, computed over a **rolling window** (say the last 50 days). Correlation tells you how *similar* two instruments' movements are; from that you build a **distance**: highly correlated instruments are "close," uncorrelated ones are "far." Now every instrument is a point in a space where distance means "how differently these two move." That collection of points is the **point cloud** for that window. As the window rolls forward day by day, you get a *sequence* of point clouds — a movie of the market's shape evolving.

4.6.3 Finding the holes (Vietoris–Rips and persistent homology)

Now the clever part: how do you detect "holes" and "clumps" in a cloud of points? You can't see them directly. The trick is to **grow balls** around every point and watch what happens.

Imagine putting a small sphere around each point and slowly inflating all of them at the same rate. When two spheres touch, you draw a connection between those points. As the spheres grow, isolated points link up into clusters, clusters merge, and sometimes a *ring* of connections encloses an empty region — a **loop** — which later fills in as the spheres keep growing. This growing scaffold of connections is called a **Vietoris–Rips complex**, and the single growing radius is often called the *filtration parameter*.

Persistent homology is the bookkeeping of this whole process. It records, for every feature, the radius at which it was *born* and the radius at which it *died*:

- **Connected components** (separate clusters) — how many distinct pieces there are, and when they merge. In topology-speak these are the "dimension-0" features, and they're written **Betti-0**.
- **Loops** (one-dimensional holes, like the hole in a ring) — when a loop forms and when it fills. These are **Betti-1**, the ones most watched in market work, because a loop closing/opening reflects a change in how assets circle around each other.
- **Voids** (enclosed empty volumes, like the inside of a hollow ball) — **Betti-2**, if you go that far.

The word *persistent* is the key insight: a feature that is born and dies almost immediately (born at radius 0.10, dies at 0.11) is just **noise** — a random accident of where the points fell. A feature that **persists** across a wide range of radii (born at 0.10, still alive at 0.60) is a *real*, robust piece of structure. Persistence is how you separate signal from noise automatically, without hand-tuning a threshold.

The standard way to *draw* this is a **persistence diagram**: each feature is a dot, with its birth radius on one axis and death radius on the other. Dots far from the diagonal (big gap between birth and death) are the meaningful, persistent structures; dots hugging the diagonal are noise.

4.6.4 Turning shape into a single risk number

A persistence diagram is a scatter of dots — powerful, but not something you can trend on a dashboard or feed to a model easily. So QuantSim converts it into a stable numerical form called a **persistence landscape**, which turns that scatter of dots into a set of tidy functions you can average, compare and do statistics on. From the landscape and the Betti numbers, the platform computes a single **composite topological risk score** — a rolled-up "how structurally stressed is the market right now" number that can sit on a chart next to volatility and be trended over time.

4.6.5 Catching the moment it changes (regime-transition detection)

Because the analysis runs on a *rolling* window, you get one topological summary per window — a sequence of them through time. **Regime-transition detection** compares *consecutive* windows and flags when the shape changes sharply: yesterday's structure and today's structure are suddenly very different. That jump is the early-warning signal — the hypothesis being that the *shape* lurches before the *averages* do.

4.6.6 The whole pipeline in one line

Putting the seven stages in order, so you can recite it: **rolling correlation → distance matrix → Vietoris–Rips complex → persistent homology → Betti numbers → persistence landscape → composite risk score**, with regime-transition detection comparing one window's result to the next. That single sentence *is* the intellectual core of your project.

4.6.7 The honest caveat

Say this part plainly and you'll sound like a serious researcher, not a salesperson: TDA is a **promising, research-grade early-warning signal, not a proven crystal ball.** The academic evidence (notably work by Gidea and Katz on topological indicators before the 2000 and 2008 crashes) is encouraging but limited. QuantSim's contribution is to *operationalise* this — to build it into a running system and study it — not to claim it definitely predicts crashes. Framing it as an honest hypothesis under test is both more defensible and more impressive than overclaiming.

4.7 Lens three: machine learning, done carefully

The third lens is **machine learning (ML)** — using algorithms that learn patterns from historical data to make predictions. QuantSim's ML produces a specific set of outputs: a **sentiment score** for each news article (is this good or bad news, and how strongly), a **volatility forecast** (how jumpy will things be), a **market-regime classification** (are we in a calm or turbulent state), a **drawdown-risk probability** (how likely is a serious fall), an overall **portfolio-health score**, and standard **strategy metrics** like the Sharpe ratio (return earned per unit of risk taken).

The specifications are deliberately careful to say *what* to predict without over-committing to *which* algorithm — whether a given forecast uses gradient-boosted trees, a random forest, an LSTM neural network, or a transformer for the news is treated as a design decision, not a fixed requirement. This is good engineering discipline (you keep the option open) and a good defense posture (you're specifying outcomes, not marrying yourself to a technique).

Two pieces of supporting machinery matter more than the choice of algorithm, and they're where the ML lens earns its credibility.

The model registry. Every trained model is versioned and recorded, so any prediction the system ever made can be traced back to the exact model version that produced it. Analogy: it's a library catalogue for models — nothing is an anonymous black box floating around; each has an ID, a version, and a record of how it was trained. This is what makes results reproducible and auditable.

The point-in-time feature store. This is the subtle, important one, and §4.8 is devoted to it because it's the difference between honest ML and accidentally cheating.

4.8 The subtle trap: look-ahead bias and the feature store

Here is the single most important correctness idea in the ML lens, and it's the kind of thing panels love to probe.

When you train a model on historical data to predict the future, it is terrifyingly easy to accidentally let the model "see the future" during training. This is called **look-ahead bias**. A classic example: you're predicting Monday's volatility, and one of your inputs is a number that was actually only finalised on Tuesday. In your neat historical table it *looks* like it was available on Monday, so the model happily uses it — and reports fantastic accuracy that completely evaporates in the real world, because on a real Monday that number doesn't exist yet. The model wasn't smart; it was cheating, and you didn't notice.

QuantSim makes this structurally impossible with a **bi-temporal point-in-time feature store**. "Bi-temporal" means every stored fact carries **two** timestamps, not one:

- the **event time** — when the thing actually happened in the world (Monday's trading);
- the **knowledge time** — when *we first knew* that fact (when it landed in our system).

Now, when the model asks for "everything known *as of* Monday," the store returns only rows whose *knowledge time* is Monday or earlier. The Tuesday-finalised number is automatically excluded, because we didn't know it yet on Monday. Analogy: it's like a diary where every entry is stamped with the date you *wrote* it, not just the date it's *about* — so you can faithfully reconstruct "what did I actually know on this date," with no hindsight leaking in. This is what makes QuantSim's backtests trustworthy, and it's a genuinely sophisticated thing to have designed in from the start.

4.9 Making it all explainable: SHAP and the LLM narration

The final wrap-around capability is **explainability**, and it comes in two layers.

The first layer is **SHAP** (SHapley Additive exPlanations). When a model makes a prediction, SHAP tells you *how much each input pushed the answer up or down*. Say the drawdown-risk model outputs 70%. SHAP might report: rising volatility pushed it +25%, a spike in negative news sentiment +15%, worsening correlations +10%, and so on, all adding up to explain the 70%. The underlying idea is borrowed from cooperative game theory — Shapley values, which are a mathematically fair way to divide "credit" among players for a joint outcome; here the "players" are the input features and the "payout" is the prediction. The upshot is you can always answer "*why* did the model say that?" with specific, quantified reasons rather than a shrug.

The second layer is **natural-language narration** via a **large language model (LLM)**. SHAP gives you a technical breakdown; the LLM turns that into a sentence an ordinary person can read: "Drawdown risk is elevated at 70%, driven mainly by rising volatility and a surge of negative news sentiment over the past week." Crucially, QuantSim is designed so that if the LLM is slow or unavailable, it **falls back** to a templated explanation built directly from the SHAP numbers — so the feature degrades gracefully to a plainer sentence rather than failing outright. Explanations are also **cached**, so asking the same question twice doesn't re-run the model. This "always give an answer, prefer the nice one, never hang waiting for it" design is a recurring theme you'll see again in Part 5.

Part 5 — How it's built, in easy words

Part 3 told you *what* the system does; this part tells you *how it's put together* as software. The goal is that you can defend the architecture, not just the features. Every idea here is explained plainly first, with the proper term attached.

5.1 One big program vs. many small ones

The first big decision any system like this faces: do you build **one big program** that does everything (a **monolith**), or **many small programs** that each do one job and talk to each other (**microservices**)?

A monolith is like a **single all-in-one machine**: simpler to start, everything's in one place, but if one part jams the whole machine stops, and you can't upgrade the toaster without taking down the fridge. Microservices are like a **kitchen full of separate appliances**: more wiring to set up, but each can be replaced, upgraded, or scaled on its own, and one breaking doesn't kill the rest.

QuantSim is built as **microservices**, and the reason is specific to this project: the workloads are wildly different from each other. The Monte Carlo simulation is a CPU-hungry number-cruncher; the market-data service is a steady, memory-light streamer; the topology pipeline runs in heavy scheduled batches. If they lived in one program you'd have to scale all of them just because one was busy. Splitting them lets you give the simulation engine lots of workers while the login service stays small. It also means one team member can own a service end to end. The honest trade-off — which you should acknowledge, not hide — is that microservices add real complexity in *communication* and *keeping data consistent*, and much of Part 5 is about how QuantSim tames exactly that complexity.

5.2 The eight services (and the exact counting)

QuantSim is made of **eight services**, plus one **API gateway** in front of them. Learn this list and the vocabulary that goes with it, because precise counting is something a panel can easily test.

Short name	Service	Its one-line job
M0	Platform & Identity	Accounts, the four roles, login, permissions, audit trail
M1	Market Data	Ingest + clean price/news/reference data; own market environments
M2	Simulation	Fit stochastic models and generate Monte Carlo markets
M3	Trading	Event-driven order matching and the append-only trade log
M4	Machine Learning	Predictions, model registry, point-in-time feature store
M5	Risk & TDA	Stress testing **and** the topology pipeline (the CCP)
M6	Explainability	SHAP attributions + LLM plain-English narration
M7	Dashboard & Alerting	The single screen over everything + the alert engine

Now the counting vocabulary — say these numbers exactly:

- **Eight services** — the eight rows above. Never call it "nine services"; the ninth deployable thing is the *gateway*, which is not a domain service.
- **Eight databases** — one private database per service (the next section explains why).
- **Nine deployable units** — the eight services plus the API gateway.
- **Seven worker deployments** — background job/worker processes (explained in §5.7), separate from the request-handling parts of the services.
- **Sixteen Kubernetes Deployments** — when you count every separately-scalable running piece in the full cluster (service APIs, workers, gateway, and supporting pieces), it comes to sixteen.
- **Four roles** — Viewer, Analyst, Quant, Admin.

The mental model: *eight services, each with its own database, deployed (with the gateway) as nine units, which in the full cluster expand to sixteen separately-scalable Deployments including seven background-worker ones.*

5.3 Why every service has its own database

The rule is **database-per-service**: each of the eight services owns a **private** database that no other service is allowed to touch directly. There are no shared tables, and no service reaches into another's database to run a join or follow a foreign key. If M5 (Risk) needs something M3 (Trading) owns, it must **ask M3 through its API** or **listen for an event M3 publishes** — never read M3's tables.

Why be this strict? Because the moment two services share a table, they're secretly welded together: you can't change that table's shape without coordinating both, you can't scale or deploy them independently, and a bad query in one can lock up the other. Analogy: it's like **each flat in a building having its own locked fridge** rather than one communal fridge — nobody can quietly reorganise or spoil what's yours, and you can renovate your kitchen without asking the whole building. The cost is that when M5 wants M3's data it has to request or subscribe to it rather than just grabbing it — which is exactly the coordination problem the next sections solve.

5.4 Two ways services talk: asking vs. announcing

Services communicate in two fundamentally different styles, and knowing when each is used is a core piece of understanding.

Synchronous request/response — "asking." One service calls another and *waits* for the answer, like a phone call. This is used when the caller genuinely needs the reply *right now* to continue — for example, the gateway asking M0 "is this login valid?" The downside is coupling in time: if the called service is slow, the caller waits, and delays can cascade. So QuantSim uses synchronous calls sparingly and wraps them in the safety net described in §5.8.

Asynchronous events — "announcing." A service does its work, then publishes a message saying "this happened" — and *doesn't wait* for anyone. Other services that care are subscribed and react in their own time, like posting to a group chat rather than phoning each person. This is the *primary* integration style in QuantSim, and the tool that carries these messages is **Apache Kafka**, a durable, ordered log of events that every environment runs.

There are three broad families of events flowing over Kafka: high-volume **streaming** messages (price ticks, orders) for the live simulation; coarser **domain events** ("a simulation finished," "a stress test completed") that announce something meaningful happened; and **read-model feeds** that keep other services' local copies up to date (see §5.6). The big advantage of the announcing style is **decoupling**: M2 doesn't need to know that M5, M4, and M7 all care that a simulation finished — it just announces it, and whoever's interested reacts. You can add a new listener later without touching M2 at all.

5.5 Making "announcing" reliable: outbox, idempotency, and sagas

Announcing is great until you hit the hard question: *what if the service saves its work but crashes before it manages to announce it?* Then the world is inconsistent — the trade happened but nobody was told. QuantSim solves this trio of problems with three well-known patterns.

The transactional outbox solves "save and announce must both happen or neither." Instead of saving the work and *then* separately sending the event (two steps that can fail in between), the service writes the event into a special **outbox table inside its own database, in the same transaction as the work itself.** Since it's one database transaction, either both the work and the outbox row commit, or neither does — they can't get out of sync. A separate little process then reads the outbox and publishes the events to Kafka, marking each as sent. Analogy: rather than doing a task and hoping you remember to post about it, you write the task and the sticky-note "to post" in the same pen-stroke; a helper later mails every sticky note. Nothing done is ever left unannounced.

Idempotency solves "what if the same event gets delivered twice?" Kafka guarantees *at-least-once* delivery, which means occasionally you'll see a duplicate. A consumer is **idempotent** if processing the same event twice has the same effect as processing it once — usually by remembering the IDs it has already handled and ignoring repeats. Analogy: a turnstile that's already counted your ticket won't count it again if you wave it a second time. This makes the whole event system safe against the duplicates that *will* happen.

Sagas with compensation solve "how do we complete a multi-service operation when we can't use one big database transaction across services?" A **saga** is a sequence of local steps, each in a different service, stitched together by events. If a later step fails, you can't just "roll back" the way a single database would — the earlier steps already committed in other services. So instead each step has a **compensating action** that undoes its effect. Analogy: booking a holiday — flight, then hotel, then car. If the car booking fails, you don't have a magic undo button; you run *compensations*: cancel the hotel, cancel the flight. QuantSim uses **choreographed** sagas (each service reacts to events and knows its own next step and its own compensation) rather than a central conductor, which keeps services independent.

5.6 Local copies done safely: read models and staleness

Because no service may read another's database (§5.3), a service that frequently needs a little of another's data keeps its **own local copy**, called a **read model**, kept fresh by listening to events. For example, the dashboard service keeps a lightweight local summary of portfolios so it can render instantly without phoning the Trading service on every page load.

The disciplined rule around this: a read model is **never treated as the source of truth** — it's a convenience cache. The authoritative copy always lives in the owning service. And because events take a moment to arrive, a read model can be slightly behind reality; QuantSim explicitly *measures* this lag (called **staleness**) rather than pretending it's zero. Analogy: your read model is like a **printed timetable** — usually right,

occasionally a little out of date, and you always know the live board at the station is the real authority. Being upfront that read models are eventually-consistent, and that staleness is monitored, is a mature design stance.

5.7 The special memory of the Trading engine: event sourcing

The Trading service (M3) does something more rigorous than the others, because in trading the *history* is sacred — you must be able to prove exactly what happened and in what order. It uses **event sourcing**.

In an ordinary system you store the *current state* — "this portfolio holds 100 shares" — and overwrite it as things change. Event sourcing flips that: you store the **complete, append-only list of everything that ever happened** — "bought 60, bought 60, sold 20" — and the current holding of 100 is *computed* by replaying that list. You never edit or delete; you only ever append. Analogy: it's a **bank passbook**, not a sticky note with today's balance — every transaction is written down forever, and the balance is just the running total. The order is preserved per portfolio (each portfolio has its own strictly-ordered sequence), so you can **replay any portfolio's entire life exactly**, step by step — which is precisely the "perfect replay" feature from Part 3. It also gives you a natural, tamper-evident audit trail for free, because the log *is* the history.

5.8 When things break: the resilience chain

In a system of many services talking over a network, failures are normal, not exceptional — a service will occasionally be slow or down. QuantSim wraps risky calls in a standard **chain of protections**, each guarding against a different failure:

A **timeout** means "don't wait forever" — give up on a call that's taking too long. A **bounded retry with jitter** means "try again a few times, but not infinitely, and stagger the attempts" so a hiccup is smoothed over without everyone retrying in lockstep and creating a stampede. A **circuit breaker** means "if a service is clearly down, stop hammering it" — after enough failures, calls fail fast instantly for a while, giving the sick service room to recover, then cautiously test if it's back. A **bulkhead** means "isolate the failure" — like the watertight compartments in a ship, one flooded section can't sink the whole vessel, so one overwhelmed dependency can't consume all the system's resources. And when all else fails, a **named fallback** provides a sensible degraded answer rather than an error — this is the same "graceful degradation" you saw with the LLM narration falling back to a template.

The philosophy tying these together is **graceful degradation**: when something breaks, the system should get *worse in a small, controlled way* (a slightly stale number, a plainer explanation, a "try again shortly") rather than collapsing. A defense-ready way to say it: "we assume dependencies will fail, and we've designed each interaction to fail small."

5.9 Keeping it secure and accountable

Security in a microservices system can't just live at the front door, because there are many services behind it. QuantSim's approach has a few plain principles.

When you log in, M0 issues a **token** (a signed digital pass, a **JWT**) that proves who you are. Every service can check that pass is genuine **on its own**, by verifying the signature against M0's published public key — it doesn't have to phone M0 for every request. Analogy: it's a **tamper-proof wristband** at an event; any staff member can see it's real without radioing the box office. Crucially, *authorisation* — deciding whether you're actually allowed to do this specific thing to this specific data — is enforced **inside the service that owns the**

data, and it **fails closed** (if in doubt, deny). That's why the row-level scoping from Part 2 can't be bypassed by taking an odd route in: the check lives where the data lives.

Beyond that: each service holds its **own credentials** (fetched from a secrets manager, Vault, not hard-coded), traffic is encrypted in transit (**TLS**) everywhere, and every service keeps an **audit trail** recording who changed what and when — including the *before* value when something is edited or deleted, so nothing vanishes without a trace.

5.10 Background work: jobs and workers

Not everything can happen while a user waits. A Monte Carlo run or a full topology pipeline takes real time, so QuantSim splits work into two kinds. **Request-handling** parts answer the user quickly ("okay, your simulation is queued"). **Background workers** do the heavy lifting off to the side and announce when they're done. This is why there are **seven worker deployments** among the sixteen Kubernetes Deployments — dedicated processes for the long-running jobs (simulation sharding, the TDA batch, ML training, and so on) that pull work from queues and can be scaled up independently when there's a backlog. Analogy: the counter staff take your order instantly; the kitchen (workers) cooks in the back and calls your number when it's ready.

5.11 Where the difficulty *actually* lives (and why that's the right story)

A useful framing for the whole architecture: the microservices design is **a response to the problem, not the star of the show.** The genuinely hard, original parts of QuantSim are the **analytics** (the topology pipeline and the risk mathematics) and the **correctness edges** (no look-ahead bias, exact replay, money handled to the right precision, alert-once semantics). The architecture exists to *serve* those — to let the heavy analytics scale, and to protect the correctness guarantees. If a panellist asks "isn't microservices overkill for a student project?", the honest and strong answer is: the split is justified by the genuinely different workloads and the correctness isolation, and the intellectual weight of the project lives in the analytics and correctness, with the architecture as a considered supporting response.

Part 6 — End-to-end workflows (what actually happens when…)

Now the payoff. This part traces real user actions all the way through the system, so you can see the services, events, and patterns from Parts 3–5 working *together*. Each walkthrough names which service does what, so you could point at the architecture diagram while narrating. Steps are numbered because these genuinely are sequences where order matters.

6.1 Logging in and creating a portfolio

4. You open QuantSim and enter your credentials. The request hits the **API gateway**, which forwards it to **M0 (Platform & Identity)**.
5. M0 checks your credentials, and if they're valid, issues you a signed **token (JWT)** that encodes who you are and your role (say, Quant).
6. You ask to create a portfolio. The gateway routes the request — carrying your token — to **M3 (Trading)**, which owns portfolios.
7. M3 verifies your token *itself* against M0's public key (no phone call needed), then checks authorisation: does a Quant have permission to create a portfolio? Yes. It **fails closed** if anything is off.

8. M3 creates the portfolio. Because M3 is **event-sourced**, this isn't an overwrite — it appends a "portfolio created" event to that portfolio's append-only log, and in the *same database transaction* writes a matching row to its **outbox**.
9. The outbox publisher picks up that row and announces a "portfolio created" **domain event** on Kafka. M3 is done; it doesn't wait for anyone.
10. **M7 (Dashboard)** is subscribed. It hears the event and updates its local **read model** of portfolios, so the new portfolio appears on your dashboard — without M7 ever touching M3's database.

The teaching point: even this simple action shows token-based auth verified locally, authorisation in the owning service, event sourcing, the outbox, and a read model updated by an event.

6.2 Running a simulation

11. As a Quant, you configure a run: pick an instrument, choose a stochastic model (say jump-diffusion), and set how many Monte Carlo paths.
12. The request goes to **M2 (Simulation)**. It first **calibrates** the model — fitting its knobs to the instrument's real history it obtained from **M1 (Market Data)** — and enforces the rule that the model is valid for that instrument.
13. Because generating thousands of paths is slow, M2 doesn't make you wait. It **queues** the job and immediately replies "simulation started." This is the request-handler/worker split.
14. The **simulation workers** (some of the seven worker deployments) pick up the job and generate the paths. Since Monte Carlo paths are **embarrassingly parallel**, the work is split across workers running at the same time.
15. As paths complete, results are assembled. When the whole run is done, M2 writes the results to its own database and — via its **outbox** — announces a "simulation completed" domain event on Kafka.
16. Interested services react: **M7** updates your dashboard so the finished run appears, and any lens that wants to consume the synthetic market (Risk, ML) now can.

The teaching point: the queue-and-worker pattern, embarrassingly-parallel Monte Carlo, calibration validity, and "announce when done."

6.3 Running a stress test (a saga across services)

This one shows a **saga**, because a stress test needs several services to cooperate and you can't wrap them in one database transaction.

17. As an Analyst, you launch a "flash crash" stress test on a portfolio. The request goes to **M5 (Risk & TDA)**.
18. M5 needs the portfolio's current holdings, which **M3** owns, and the relevant market/price data, which **M1** owns. It obtains each through the proper channel (an API call or its event-fed read model) — never by reading their databases.
19. M5 applies the flash-crash scenario to those holdings and computes the outcomes: maximum loss, **VaR** and **CVaR** under the scenario, recovery time, the worst-contributing instruments, and concentration.
20. M5 saves the stress result and announces a "stress test completed" event via its outbox.
21. **M6 (Explainability)** hears it and produces an explanation — using SHAP-style attribution to say *which* holdings drove the loss, then the **LLM** to write a plain-English summary (falling back to a template if the LLM is unavailable).
22. **M7** updates the dashboard with the result and its explanation.

23. **If a step fails partway** — say M5 can't get the data it needs — the saga runs **compensating actions** to leave everything tidy (mark the stress run as failed, undo any partial state), rather than leaving a half-finished result lying around.

The teaching point: a choreographed saga with compensation, cross-service data obtained the right way, and explainability layered on afterward via events.

6.4 Viewing topological risk and catching a regime change

24. On a schedule (or on request), a **TDA worker** in **M5** runs the topology pipeline over a **rolling window** of recent market data from **M1**.
25. It executes the seven stages in order: rolling **correlation** → **distance matrix** → **Vietoris–Rips complex** → **persistent homology** → **Betti numbers** → **persistence landscape** → **composite topological risk score**.
26. That score is stored as the latest point in a time series, and announced as an event.
27. The pipeline runs again for the next window, producing the next score. **Regime-transition detection** compares this window to the previous one; if the shape has changed sharply, M5 flags a possible **regime transition**.
28. **M7** shows you the topological-risk view — the score trending over time next to volatility — and highlights any flagged transition. If you'd set an alert on it, the alert engine fires (see §6.6).
29. If you click "why," **M6** narrates what changed in the structure in plain language.

The teaching point: this is the project's headline capability shown end to end — the full pipeline, the rolling-window sequence, transition detection, and explanation.

6.5 Getting an ML prediction and its explanation

30. You ask **M4 (Machine Learning)** for a drawdown-risk prediction on a portfolio.
31. M4 gathers the model's inputs from its **point-in-time feature store**, asking only for what was known **as of** the relevant moment — so no **look-ahead bias** can sneak in.
32. M4 runs the appropriate model version (tracked in the **model registry**, so the prediction is traceable) and returns, say, "70% drawdown risk."
33. You ask for the reasoning. **M6** computes **SHAP** attributions — rising volatility +25%, negative news sentiment +15%, and so on — then the **LLM** turns that into a readable sentence, cached for next time and templated if the LLM is down.
34. **M7** displays the number alongside its explanation.

The teaching point: the two pieces of ML machinery that give it credibility (feature store preventing look-ahead, registry giving traceability), plus the two-layer explanation.

6.6 An alert firing — exactly once

35. Earlier, you set an alert rule in **M7**: "tell me if topological risk crosses 0.8."
36. M7's **alert engine** continuously watches the relevant events/values as they arrive.
37. The topological-risk score crosses 0.8. The condition becomes true, so M7 **pushes you a notification** immediately.
38. Over the next several windows the score stays *above* 0.8 — the condition is *still* true, event after event. The engine does **not** notify you again; it tracks that this condition is already in an "alerting" state and stays quiet.

39. Later the score drops back below 0.8 and the condition **resolves**. Only if it crosses again will a fresh alert fire.

The teaching point: **alert-once-per-condition, not once-per-event.** This is one of the "correctness edges" from §5.11 — a small but genuinely tricky bit of state management that prevents the notification flood which would make alerts useless. Being able to explain *why* naïve alerting (fire every time the value is over the line) is wrong, and how QuantSim avoids it, is a strong detail to have ready.

Part 7 — The technology stack, and why each piece

Here's the toolkit, grouped by what it's for, with a plain reason for each choice. You don't need to defend every version number, but you should be able to say *why* a tool is there.

Layer	Technology	Why it's used, in one line
Backend language	Python 3.12	The default language of quantitative finance and machine learning — every library you need already lives here
Web framework	FastAPI (async)	Fast, modern, `async` by default so a service can juggle many waiting calls without extra threads
Frontend	React + TypeScript	Component-based UI for the dashboards and charts; TypeScript catches mistakes before they ship
Database	PostgreSQL 16	A rock-solid relational database; one private instance per service
Time-series extension	TimescaleDB	Adds "hypertables," compression, and rollups to Postgres so it handles huge streams of time-stamped price/tick data efficiently
DB access	SQLAlchemy (async) + asyncpg + PgBouncer	The toolkit that lets Python talk to Postgres asynchronously, with PgBouncer pooling connections so the database isn't overwhelmed
Schema migrations	Alembic (per-service, expand/contract)	Versioned, safe database schema changes — "expand/contract" means you can change shape without downtime
Events / messaging	Apache Kafka	The durable, ordered log that carries every event between services (the "announcing" backbone)
Caching / warm store	Redis	Fast in-memory store for hot lookups and the warm data tier
Background jobs	Arq / Celery queues	The queues and workers that run the heavy jobs (simulation, TDA, training) off the request path
Topology	Ripser, GUDHI, persim	The specialist libraries that actually compute persistent homology and persistence landscapes — the TDA engine

Layer	Technology	Why it's used, in one line
Machine learning	scikit-learn, XGBoost, PyTorch, Hugging Face	The spread from classic models (scikit-learn, XGBoost) to deep learning (PyTorch) to news/NLP models (Hugging Face)
Cold storage	Parquet on S3 / MinIO	Cheap, compressed columnar files for long-term historical data you don't need instantly
Secrets	Vault	Central, secure home for each service's credentials — nothing hard-coded
Packaging / runtime	Docker + Kubernetes	Docker packages each service; Kubernetes runs, scales, and heals the sixteen Deployments
Local dev	Docker Compose	Spins the whole system up on one laptop for development and the demo
Infrastructure as code	Terraform	Describes the cloud setup as versioned files, so environments are reproducible

The three-tier data story is worth stating as its own idea, because it recurs: **hot** data (needed instantly) lives in PostgreSQL + TimescaleDB; **warm** data (needed fast but not authoritative) lives in Redis; **cold** data (rarely touched history) lives as Parquet files in object storage. Data ages from hot to cold, which keeps the fast stores small and the storage bill sane.

Part 8 — The numbers at a glance

A single sheet of the figures worth having memorised, so you're never caught out by a counting question.

Number	What it counts
8	Services (M0–M7)
8	Databases (one private per service)
9	Deployable units (the 8 services + the API gateway)
7	Background worker deployments
16	Kubernetes Deployments in the full cluster (every separately-scalable piece)
4	User roles (Viewer, Analyst, Quant, Admin)
4	Stochastic models (GBM, jump-diffusion, Ornstein–Uhlenbeck, regime-switching)
3	Risk lenses (stress testing, TDA, ML)

Number	What it counts
7	Stages in the TDA pipeline (correlation → distance → Vietoris–Rips → persistent homology → Betti → landscape → risk score)
81	Functional requirements in total
64	of those that are "Core" (committed, demonstrable)
50	Active non-functional requirements (51 issued, 1 withdrawn)
20	Architecture Decision Records (AD-1 … AD-20)
5	Stress scenario types (flash crash, liquidity shock, volatility spike, sector collapse, correlated failure)

Two phrasing traps to avoid: it's **eight services, not nine** (the ninth *deployable unit* is the gateway); and the ML method families (XGBoost, LSTM, transformers, etc.) are **design choices, not fixed requirements** — the requirements say *what* to predict, not *which* algorithm.

Part 9 — Glossary (skim this the night before)

Plain one-line definitions for every term worth knowing. In rough order of appearance.

Market environment — a world a portfolio lives inside; either a replay of real history or a synthetic simulation.

OHLCV — the five numbers describing a price bar: open, high, low, close, volume.

Return — the percentage change in price from one period to the next; puts all assets on a comparable footing.

Volatility — how jumpy returns are (their standard deviation); the everyday measure of risk, but a *late* signal.

Correlation — how much two assets move together, from +1 (lockstep) to −1 (opposite); the basis of diversification.

Stochastic model — a recipe that generates a random-but-realistic price path. QuantSim has four.

GBM (Geometric Brownian Motion) — the smooth baseline model: steady drift plus proportional random wobble.

Jump-diffusion — GBM plus occasional sudden jumps, to model news shocks and gaps.

Ornstein–Uhlenbeck — a mean-reverting "spring" model, pulled back toward a long-run average.

Regime-switching — a model that flips between market "moods" (calm vs. crisis), each with its own behaviour.

Calibration — fitting a model's knobs to a real instrument's history so the fake market resembles the real one.

Monte Carlo simulation — learning the range of outcomes by running a random process many thousands of times.

Embarrassingly parallel — work whose pieces are independent, so they can all run at the same time (Monte Carlo paths).

Stress testing — deliberately staging a disaster scenario and measuring the damage; forward-looking by design.

VaR (Value at Risk) — the loss level the worst 5% of days exceed; marks the door to the bad room.

CVaR (Conditional VaR / Expected Shortfall) — the *average* loss *given* you're in that worst 5%; looks inside the bad room.

Drawdown — the drop from a peak to a later trough; maximum drawdown is the worst such fall.

Risk concentration — how dangerously bunched your risk is (one asset, sector, or hidden common factor).

Topology — the maths of shape — what survives stretching and bending (a mug and a doughnut are "the same").

Point cloud — the market drawn as points, positioned so assets that move together sit close together.

Vietoris–Rips complex — the scaffold of connections you get by growing balls around every point until they touch.

Persistent homology — the bookkeeping of which shape features (clusters, loops, voids) are born and die as the balls grow.

Betti numbers — counts of shape features: Betti-0 = separate pieces, Betti-1 = loops, Betti-2 = voids.

Persistence — how long a feature survives as the radius grows; long-lived = real structure, short-lived = noise.

Persistence diagram — a plot of each feature's birth vs. death radius; dots far from the diagonal are meaningful.

Persistence landscape — a tidy, stable function form of the diagram that you can average and do statistics on.

Composite topological risk score — the single rolled-up "how structurally stressed is the market" number.

Regime-transition detection — flagging when consecutive windows' shapes differ sharply; the early-warning trigger.

Machine learning (ML) — algorithms that learn patterns from historical data to make predictions.

Model registry — a versioned catalogue of trained models, so every prediction traces to an exact model version.

Feature store — the managed source of model inputs ("features").

Point-in-time / bi-temporal — storing two timestamps (when it happened, when we knew it) so models can't see the future.

Look-ahead bias — accidentally letting a model use information that wasn't actually available yet; the trap the feature store prevents.

SHAP — a fair way (from game theory) to attribute a prediction to its inputs — how much each pushed the answer up or down.

LLM (Large Language Model) — the model that narrates results in plain English, with a templated fallback.

Monolith vs. microservices — one big program vs. many small cooperating ones; QuantSim is microservices.

API gateway — the single front door that routes requests to the right service (the 9th deployable unit).

Database-per-service — each service owns a private database no other service may touch directly.

Synchronous call — asking and waiting (a phone call); used sparingly for answers needed right now.

Asynchronous event — announcing that something happened and not waiting (a group-chat post); the primary style.

Apache Kafka — the durable, ordered log that carries all events between services.

Transactional outbox — writing the event into the same database transaction as the work, so nothing done goes unannounced.

Idempotency — processing the same event twice has the same effect as once; safe against duplicate delivery.

Saga — a multi-service operation stitched from local steps, each with a compensating action to undo it if a later step fails.

Compensating action — the "undo" for a saga step, since there's no cross-service rollback.

Read model — a local, event-fed copy of another service's data for fast reads; never the source of truth.

Staleness — how far behind reality a read model is; explicitly measured, not assumed zero.

Event sourcing — storing the full append-only log of everything that happened (Trading), with current state computed by replay.

Timeout / retry / circuit breaker / bulkhead / fallback — the resilience chain: don't wait forever / try a few times / stop hammering a dead service / isolate failures / give a degraded answer.

Graceful degradation — failing in small controlled ways (stale number, plainer text) rather than collapsing.

JWT — a signed digital pass proving who you are, verifiable by each service on its own.

Fail closed — when authorisation is uncertain, deny.

Row-level scoping — the rule that your role isn't enough; you only see the specific data you've been granted, checked where the data lives.

Audit trail — a record of who changed what and when, including the *before* value on edits and deletes.

Worker — a background process that runs heavy jobs (simulation, TDA, training) off the request path.

Hot / warm / cold data — instant (Postgres+TimescaleDB) / fast (Redis) / archival (Parquet on object storage) tiers.